

7

Fluent Design System for Windows Applications

The **Fluent Design System** is a set of application design principles created by Microsoft and implemented across multiple desktop, mobile, and web platforms. The Fluent Design System for Windows is a set of controls, patterns, and styles for applications built for Windows. In fact, it is the implicit styling for all WinUI controls.

It is important to learn the tenets of Fluent Design and how to implement them in your WinUI applications. We will also explore the **Fluent XAML Theme Editor** application for Windows. This application assists developers in creating a theme for their applications, including color schemes and style elements such as borders and corners. Developers can then easily import the resources to implement the theme.

In this chapter, we will cover the following topics:

- Learning the concepts of Fluent Design
- How to find the latest information about Fluent Design
- Incorporating Fluent Design concepts into WinUI applications
- Using the Fluent XAML Theme Editor to customize and use a UI theme
- Exploring the **Acrylic** material and the Fluent Design System
- Using the **Mica** material in WinUI applications

By the end of this chapter, you will understand the Fluent Design System for Windows applications. You will also know how to incorporate these design standards into your WinUI applications.

Technical requirements

To follow along with the examples in this chapter, the following software is required:

- Windows 10 version 1803 (version 17134) or later
- Visual Studio 2022 or later with the .NET Desktop Development workload installed and configured for Windows App SDK development

The source code for this chapter is available on GitHub at this URL:

<https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/master/Chapter07>.

What is the Fluent Design System?

The Fluent Design System is a cross-platform system that helps developers create beautiful, intuitive applications. The website for Fluent Design (<https://fluent1.microsoft.design/>) has dedicated pages with resources for developers on many platforms:

- Android
- iOS
- macOS
- Web
- Windows
- Cross-platform (React Native)

NOTE

Microsoft has started releasing their Fluent 2 design guidance at <https://fluent2.microsoft.design/>. You can think of Fluent 1 styles as Windows 10 style, whereas Fluent 2 style is similar to the look and feel of Windows 11. At the time of this writing, the Fluent 2 guidance has only been published for React web applications. Other platforms are not yet available.

Fluent Design aims to be simple and intuitive. While it maintains its design philosophy across platforms, it also adapts aspects of its design to feel native on every platform. In **Chapter 1, Introduction to WinUI**, we discussed the origins of some of the current Fluent Design concepts in the **Metro** design that was introduced with Windows Phone. While the look and feel of Microsoft's designs have evolved over the years, some of the principles remain. The three core principles of the Fluent Design System are the following:

- **Natural on every device:** Software should adapt to the device where it's running, whether it's a PC, tablet, game console, phone, or AR/VR device
- **Intuitive and powerful:** The UI anticipates users' actions and pulls them into the experience while using the app
- **Engaging and immersive:** The design pulls from real-world elements, using light, shadow, texture, and depth to create an immersive experience for the user

The driving philosophy behind the design is to adapt and feel natural. The device and the app should feel comfortable and anticipate the user's actions.

This is a very abstract and high-level explanation so far. Let's explore the specifics of Fluent Design for Windows in the next section.

Exploring Fluent Design for Windows

For Windows applications, Fluent Design covers several areas. When compared to other design systems, Fluent is more all-encompassing.

Apple's **Human Interface Guidelines**

(<https://developer.apple.com/design/human-interface-guidelines/>)

have only been widely adopted on Apple's platforms: iOS, iPadOS, and macOS. Google's **Material Design** (<https://material.io/>) system has seen broader adoption but only has toolkits available for Android, Flutter, and the web.

Fluent Design is most often compared with Material Design as they share some concepts when it comes to shapes and texture, but Fluent Design uses transparency to much greater effect than Material Design. Fluent

Design provides a rich toolset that you can use in WinUI and nearly any other development platform.

Let's explore some of these design aspects and how they apply to your WinUI applications.

Controls

A control equates to a single element of *user input* or *interaction*. We have already explored many of the controls available in WinUI in [Chapter 5, Exploring WinUI Controls](#). This is what some of the common WinUI controls look like in light and dark modes in Windows 11:

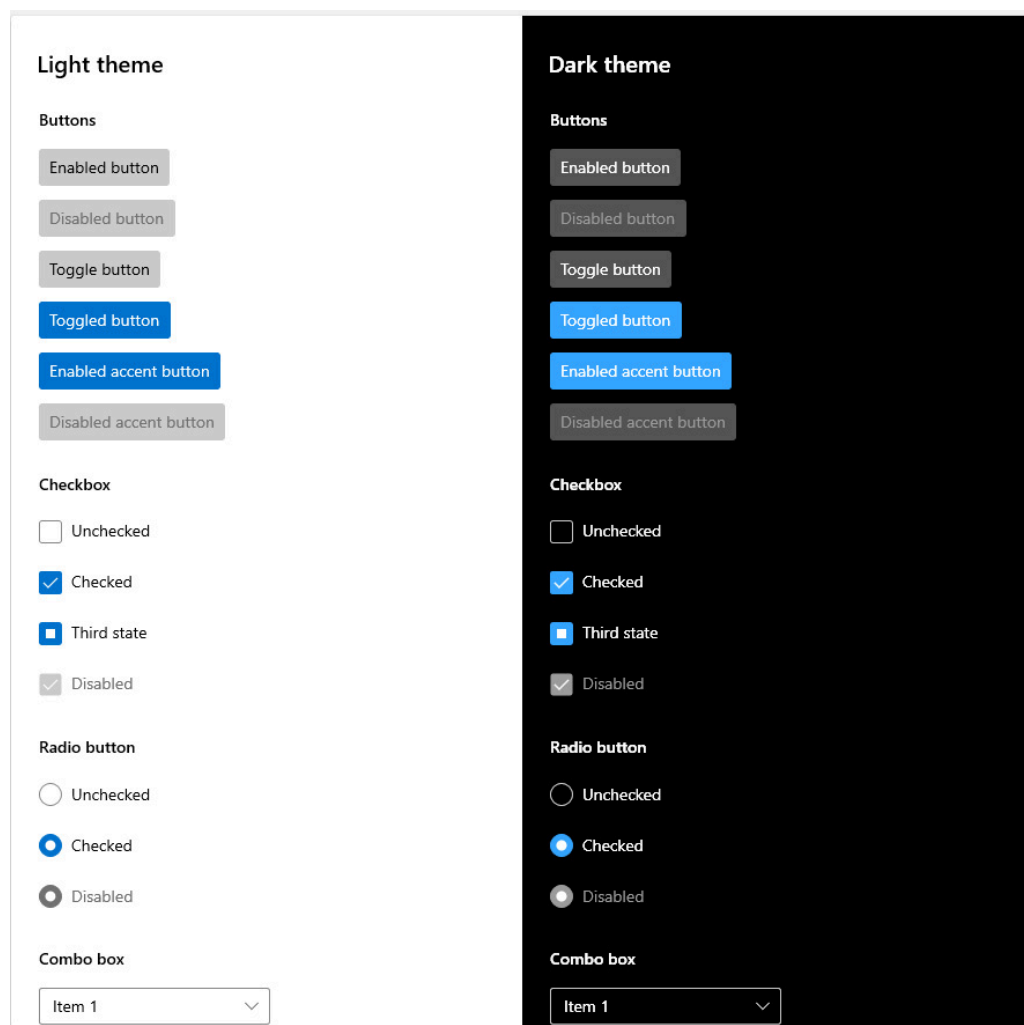


Figure 7.1 – Some common controls in light and dark modes

By default, the WinUI controls make use of Fluent styles. We will see how to override the default Fluent styles in our WinUI controls later in the chapter.

Patterns

Patterns are groups of related controls or a group of controls that becomes a single new element. This group could be added to a composite control for re-use. Some examples of patterns in WinUI include the following:

- **Search:** In its simplest form, a search pattern needs to have controls for accepting input, invoking the search, and displaying the search results. An additional element may be added for suggesting searches before any input is received. Assistants such as *Amazon Alexa* do this based on a user's calendar, contacts, news preferences, and so on. Adding an autosuggest list based on user input is a common feature of modern search controls. You could also integrate chat controls with **artificial intelligence (AI)** such as Microsoft's **Bot Framework** or OpenAI's **ChatGPT** to ask some follow-up questions based on the initial search parameters.
- **Forms:** Forms are a very common control pattern. They consist of groups of related labels, input controls, and command buttons that collect a related set of data elements. Some common forms with potential for re-use are user account creation forms and forms for collecting user feedback. Forms should follow the Fluent Design guidelines for spacing, flexible layout, and using typography for creating a hierarchy (Microsoft Learn example: <https://learn.microsoft.com/windows/apps/design/controls/forms>).
- **List/details:** The list/details control pattern is seen so frequently that we probably don't notice it most of the time. You may think our My Media Collection application follows this pattern (with the items list and a page to view and edit item details), but this pattern describes how to show these views on the same page. This is typically achieved with a **ListView** control and a **SplitView** control to separate the list from the selected item's details. Depending on the width of the page, the two views can either be stacked vertically or displayed side by side (Microsoft Learn example: <https://learn.microsoft.com/windows/apps/design/controls/list-details>).

Each of these patterns encapsulates the elements of Fluent Design to create a composite control that can be reused across applications. You may have control patterns in your projects that could be added to a shared control library for ease of reuse. Shared libraries such as these can save time and ensure that good design practices are followed across teams.

Layout

The layout is important to ensure that an application adapts to any screen size or orientation. Flexibility is a key tenet of a well-designed layout.

When a window or page is resized, the contents can adapt by repositioning controls, adding/removing items, changing the flow of items, replacing controls with others that better fit the current available space, or simply resizing items. This is typically handled in XAML with **Visual States**. You can define a different **VisualState** for each size threshold to which your page must adapt, possibly defined as **Narrow**, **Default**, **Wide**, and **ExtraWide**. Each **VisualState** updates control properties to adapt to the new layout. Microsoft Learn has a great example of this at <https://learn.microsoft.com/windows/apps/design/layout/layouts-with-xaml>. WinUI includes several different layout panels that can help developers create the right layout for their pages and respond to changes in size, orientation, and resolution.

Input

There are Fluent Design recommendations for responding to user input. There are guidelines for reacting to the traditional mouse and keyboard input that developers have been handling for decades. Modern applications can do things such as pan, zoom, rotate, or scroll based on mouse input. A keyboard may be a physical keyboard or an onscreen keyboard for mobile and touch users.

Input can come in other forms with today's hardware:

- Pen/stylus
- Touch
- Touchpad
- Gamepad/controller

- Remote control
- Surface Dial (see <https://learn.microsoft.com/windows/apps/design/input/windows-wheel-interactions>)
- AR/VR gestures
- Voice

User input can also be simulated with the input injection APIs. An example of where this might be useful is creating a **Show Me How** or **Guided Tour** feature in your app. Your code can execute some pre-defined steps, guiding the user through performing some action on the page. This API is beyond the scope of this book. To read about an example of using input injection to intercept mouse input and turn it into touch input, read this article on Microsoft Learn:

<https://learn.microsoft.com/windows/apps/design/input/input-injection>.

Style

Style encompasses multiple aspects of Fluent Design:

- **Icons:** Good icons should be simple and convey the application's purpose.
- **Color:** The color choice is important. Allowing users to customize their colors is also a great way to make your app feel personal to them. WinUI makes it easy to adapt the user's light or dark theme choice and the Windows highlight color by using theme brushes.
- **Typography:** Microsoft recommends that Windows applications all use the Segoe UI font. Selecting the font size can help convey a hierarchy within the app, such as a book or document layout. To this end, Microsoft has defined a **type ramp** (available at <https://learn.microsoft.com/windows/apps/design/style/typography#type-ramp>). A type ramp defines the increase in font size for different styled elements on the screen, such as *Body*, *Title*, and *Subtitle*. There are static resources that can be leveraged in WinUI to select the right size for a control's intended use.
- **Spacing:** Spacing between and within controls is important for readability and usability. WinUI controls allow a Standard or Compact den-

sity to be selected. More information about sizing and Fluent densities can be found here:

<https://learn.microsoft.com/windows/apps/design/style/spacing>.

- **Reveal focus:** Drawing attention to focusable elements is important for larger displays, such as an Xbox or Surface Hub. This is achieved through lighting effects with Fluent.
- **Acrylic:** This is a type of WinUI brush that creates texture with transparency. This texture gives a feeling of depth to the user interface. We will discuss Acrylic in more detail later in this chapter.
- **Mica:** This is a dynamic material like Acrylic, but unlike Acrylic it is opaque, not transparent. It creates the background of the app by incorporating elements of the current OS theme and desktop wallpaper. We will see how to incorporate Mica into your own WinUI applications later in the chapter.
- **Corner radius:** Fluent Design promotes the idea that rounded corners promote positive feelings in users. WinUI controls have a rounded corner radius consistent with Fluent Design recommendations.
- **Sound:** Sound can be an integral part of crafting an immersive experience in your apps. A subtle whooshing sound when panels slide open or closed and using just the right tone and volume in the sound of an alert can immerse the app's users.
- **Writing style:** Believe it or not, writing style is part of app design. Line-of-business applications should not have the same writing style as a casual consumer app or a puzzle game. Users will be pulled into the app experience if they don't notice the writing style at all because it fits what they expect for the application type.

These are just some of the aspects of style defined by Fluent Design. You can read more about them on Microsoft Learn:

<https://learn.microsoft.com/windows/apps/design/style/>.

Many of the aspects of Fluent style are made available to our WinUI apps via XAML styles and other static resources. Next, we will look at how we can update our sample application to respond to changes in a user's Windows theme.

Incorporating Fluent Design in WinUI applications

It is time to incorporate a few of the Fluent Design principles into the **My Media Collection** application and polish the UI a little. Most of the WinUI controls are already designed to meet Fluent standards, but there were a few properties we modified without understanding Fluent Design.

Updating the title bar

Before we even get into the XAML to improve the styles, let's fix the application's title bar. Until now, the title bar always read **MyMediaCollection** without any spaces or indication of the current page:

1. First, to fix the spacing when the app is packaged and distributed, open **Package.appmanifest** from the **Solution Explorer** window. On the page that opens, update **Display name** to **My Media Collection**. If you like, you can also change **Description**.

The screenshot shows the 'Package.appmanifest' configuration page in Visual Studio. The 'Application' tab is selected. The page contains the following fields and options:

- Display name:** My Media Collection
- Entry point:** \$targetentrypoint\$
- Default language:** en-US (with a 'More information' link)
- Description:** MyMediaCollection
- Supported rotations:** An optional setting that indicates the app's orientation preferences. Four icons are shown: Landscape (selected), Portrait, Landscape-flipped, and Portrait-flipped.
- Lock screen notifications:** (not set) (dropdown menu)
- Resource group:** (empty text box)
- Tile Update:** Updates the app tile by periodically polling a URI. The URI template can contain "{language}" and "{region}" tokens that will be replaced at runtime to generate the URI to poll. (with a 'More information' link)
- Recurrence:** (not set) (dropdown menu)
- URI Template:** (empty text box)

Figure 7.2 – Updating information in Package.appmanifest

Updating **Display name** will not change the application's title bar text in WinUI 3 as it does for UWP apps. We will have to handle that with the

AppWindow class.

2. To update the text in the app's title bar, add the following code to

MainWindow.xaml.cs:

```
using Microsoft.UI;
using Microsoft.UI.Windowing;
using Microsoft.UI.Xaml;
using System;
using WinRT.Interop;
namespace MyMediaCollection
{
    public sealed partial class MainWindow : Window
    {
        private AppWindow _appWindow;
        private const string AppTitle = "My Media Collection";
        public MainWindow()
        {
            this.InitializeComponent();
            _appWindow = GetCurrentAppWindow();
            _appWindow.Title = AppTitle;
        }
        private AppWindow GetCurrentAppWindow()
        {
            IntPtr handle = WindowNative.GetWindowHandle(this);
            WindowId windowId = Win32Interop.GetWindowIdFromWindow(handle);
            return AppWindow.GetFromWindowId(windowId);
        }
    }
}
```

3. Now, add an **internal** method named **SetPageTitle** so each page can append its title to the main window title:

```
internal void SetPageTitle(string title)
{
    if (_appWindow == null)
    {
        _appWindow = GetCurrentWindow();
    }
    _appWindow.Title = $"{AppTitle} - {title}";
}
```

4. In order for each page to access **MainWindow**, expose an internal member in **App.xaml.cs**:

```
internal Window Window => m_window;
```

5. Next, in **MainPage.xaml.cs**, add an event handler for the **Loaded** event of the page. In the event handler, add some code to append to the **Title** of the current window with the page title of **Home**. When we launch the application, the title bar should read **My Media Collection - Home**:

```
public MainPage()
{
    this.InitializeComponent();
    Loaded += MainPage_Loaded;
}
private void MainPage_Loaded(object sender, RoutedEventArgs e)
{
    var mainWindow = (Application.Current as App)?.Window as MainWindow;
    if (mainWindow != null)
    {
        mainWindow.SetPageTitle("Home");
    }
}
```

6. Finally, make the same changes in **ItemDetailsView.xaml.cs**, but set the page title to **Item Details**.

Now, when you run the application, you should see the title bar text update as you navigate between the list of items and the item details. Let's make some changes to the styles of **MainPage** next.

Changing the style of MainPage

Currently, the main page of our application doesn't have many styles. We set the **FontWeight** of a few **TextBlock** controls to **Bold** to set them apart as important items, but this doesn't follow the Fluent Design guidelines for typography. There is also a purple border separating the **ListView** header from its items:

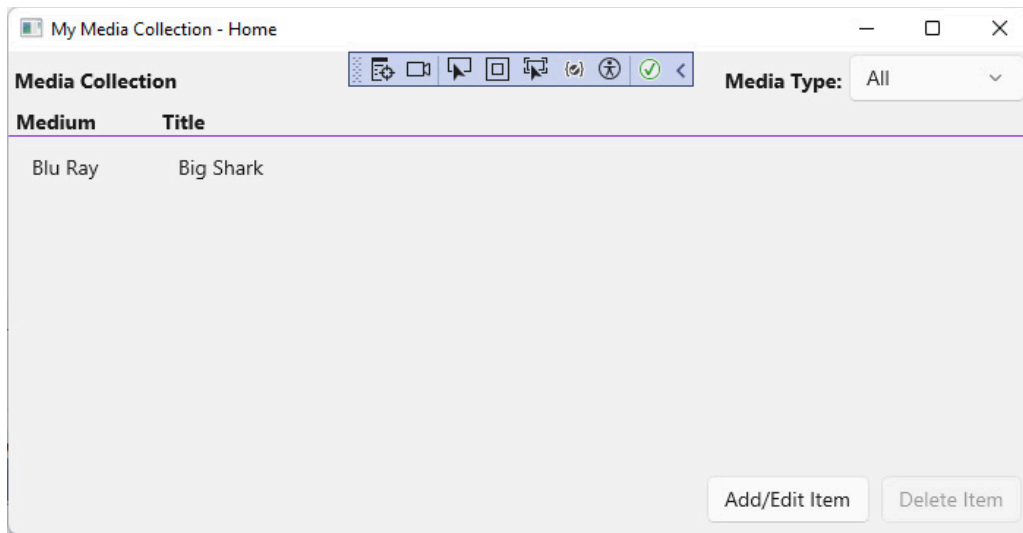


Figure 7.3 – The current main page of My Media Collection

Hard-coding colors is not a good practice. Even if you were using custom colors as product branding, you could centralize them in **Application.Resources**. Let's work our way through the **MainPage.xaml** file and make some improvements:

1. First, update the text of the first **TextBlock** from **Media Collection** to **Home**, matching the text in the window's title bar. Wrap it in a horizontally aligned **StackPanel** and add a preceding **SymbolIcon** control to display a **Home** symbol. Finally, remove the hardcoded font size and weight attributes and set the **Style** attribute to import the **SubheaderTextBlockStyle** **StaticResource**. Those changes should look like this:

```
<StackPanel Orientation="Horizontal">
    <SymbolIcon Symbol="Home" Margin="8"/>
    <TextBlock Text="Home"
        Style="{StaticResource SubheaderTextBlockStyle}"
        Margin="8"/>
</StackPanel>
```

2. We should also remove the **FontWeight** attribute from the **Media Type** label and use a Fluent style resource:

```
<TextBlock Text="Media Type:" Margin="4"
    Style="{StaticResource SubtitleTextBlockStyle}"
    VerticalAlignment="Bottom"/>
```

3. Next, change the surrounding **Grid** to a **StackPanel** and remove the **Grid.Column Definition** definitions. In addition to simplifying the layout, this will allow the **Home** symbol and text to appear above the rest of the controls on the page, reinforcing the hierarchy. The full block of code will look like this:

```
<StackPanel>
    <StackPanel Orientation="Horizontal">
        <SymbolIcon Symbol="Home" Margin="8"/>
        <TextBlock Text="Home"
            Style="{StaticResource SubheaderTextBlockStyle}"
            Margin="8"/>
    </StackPanel>
    <StackPanel Orientation="Horizontal"
        HorizontalAlignment="Right">
        <TextBlock Text="Media Type:"
            Margin="4"
            Style="{StaticResource SubtitleTextBlockStyle}"
            VerticalAlignment="Bottom"/>
        <ComboBox ItemsSource="{x:Bind ViewModel.Mediums}"
            SelectedItem="{x:Bind ViewModel.SelectedMedium, Mode=TwoWay}"
            MinWidth="120"
            Margin="0,2,6,4"
            VerticalAlignment="Bottom"/>
    </StackPanel>
</StackPanel>
```

4. Next, update the **HeaderTemplate** of the **ListView** to replace the purple **BorderBrush** attributes with **SystemAccentColor** from **ThemeResource**. This will make sure that the border's color picks up the user's preferred accent color from their selected Windows theme. Also, change each **TextBlock** to use a built-in **Style** instead of setting **FontWeight** and change the **Width** of the first column to be **120** to accommodate the larger title font:

```
<ListView.HeaderTemplate>
    <DataTemplate>
        <Grid Margin="4,0,4,0">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="120"/>
                <ColumnDefinition Width="*/>
            </Grid.ColumnDefinitions>
```

```

        <Border BorderBrush="{ThemeResource SystemAccentColor}"
            BorderThickness="0,0,0,1">
            <TextBlock Text="Medium"
                Margin="4,0,0,0"
                Style="{StaticResource TitleTextBlockStyle}"/>
        </Border>
        <Border Grid.Column="1"
            BorderBrush="{ThemeResource SystemAccentColor}"
            BorderThickness="0,0,0,1">
            <TextBlock Text="Title"
                Margin="4,0,0,0"
                Style="{StaticResource TitleTextBlockStyle}"/>
        </Border>
    </Grid>
</DataTemplate>
</ListView.HeaderTemplate>

```

Note that you will also need to change the first column **Width** to **120** in the **ListView.ItemTemplate**.

5. Finally, let's define the end of the list area by adding a border between the bottom of **ListView** and the command buttons. Do this by wrapping the buttons' **StackPanel** with a **Border** control, again using **SystemAccentColor**. **Margin= "4,0"** is shorthand that is equivalent to **Margin= "4,0,4,0"**:

```

<Border Grid.Row="2"
    BorderBrush="{ThemeResource SystemAccentColor}"
    BorderThickness="0,1,0,0"
    Margin="4,0">
    <StackPanel Orientation="Horizontal"
        HorizontalAlignment="Right">
        <Button Command="{x:Bind ViewModel.AddEditCommand}"
            Content="Add/Edit Item" Margin="8,8,0,8"/>
        <Button Command="{x:Bind ViewModel.DeleteCommand}"
            Content="Delete Item"
            Grid.Column="1"
            Margin="8"/>
    </StackPanel>
</Border>

```

6. Run the application and check out the restyled user interface. It looks much better. You can now easily see the hierarchy of data, however limited it may be in our simple application:

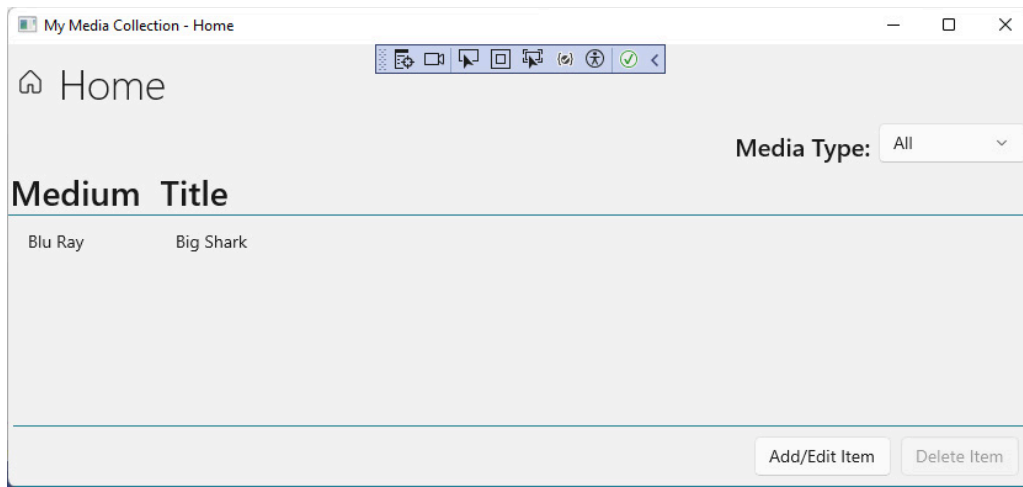


Figure 7.4 – The newly styled My Media Collection home page

Before moving on to the details page, let's see how the page looks if we select the dark mode in Windows. Open **Windows Settings**, go to **Personalization | Colors**, and select **Dark** from the **Choose your color** dropdown (if you normally use **Dark**, try changing it to **Light**):

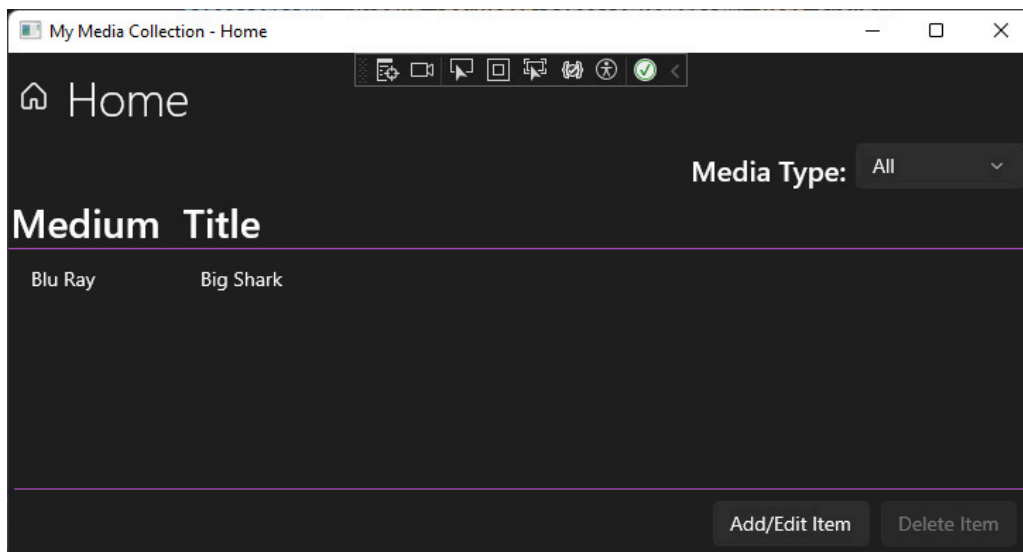


Figure 7.5 – My Media Collection running in dark mode

Everything on the page switches to dark mode without any code changes except for the title bar. To learn more about title bar customization, including changing colors or the icon, or even completely replacing it with a custom title bar, see this topic on Microsoft Learn:

<https://learn.microsoft.com/windows/apps/develop/title-bar>.

If you have a good reason to keep your application in light or dark mode, you can update **Application.xaml** to add a single attribute to the **Application** element using this command:

```
RequestedTheme="Dark"
```

This will apply the **Dark** theme to the entire application. If you have a reason to only force this theme on part of the application, the **RequestedTheme** attribute can be applied to an individual **Page** or **Control**. Now, let's apply the same types of styles to the details page.

Changing the style of ItemDetailsPage

We want to update **ItemDetailsPage.xaml** to provide it with the same overall look and feel as the main page:

1. Open the file and start by updating **Item Details**. Give it the same **Subheader TextBlockStyle** that was used on **Home** and wrap it in a horizontally aligned **Stack Panel**. Precede **TextBlock** with a **SymbolIcon**, which uses the **Edit** symbol:

```
<StackPanel Orientation="Horizontal">
    <SymbolIcon Symbol="Edit" Margin="8"/>
    <TextBlock Text="Item Details"
        Style="{StaticResource SubheaderTextBlockStyle}"
        Margin="8"/>
</StackPanel>
```

2. Next, modify the **Grid** that follows the new **StackPanel** to have top and bottom borders. Also, modify the **Margin** to have 4 px on either side of **Grid**:

```
<Grid Grid.Row="1"
    BorderBrush="{ThemeResource SystemAccentColor}"
    BorderThickness="0,1,0,1"
    Margin="4,0,4,8">
```

That's all we need to change on this page. Run the application again and navigate to the details page to see how it looks:

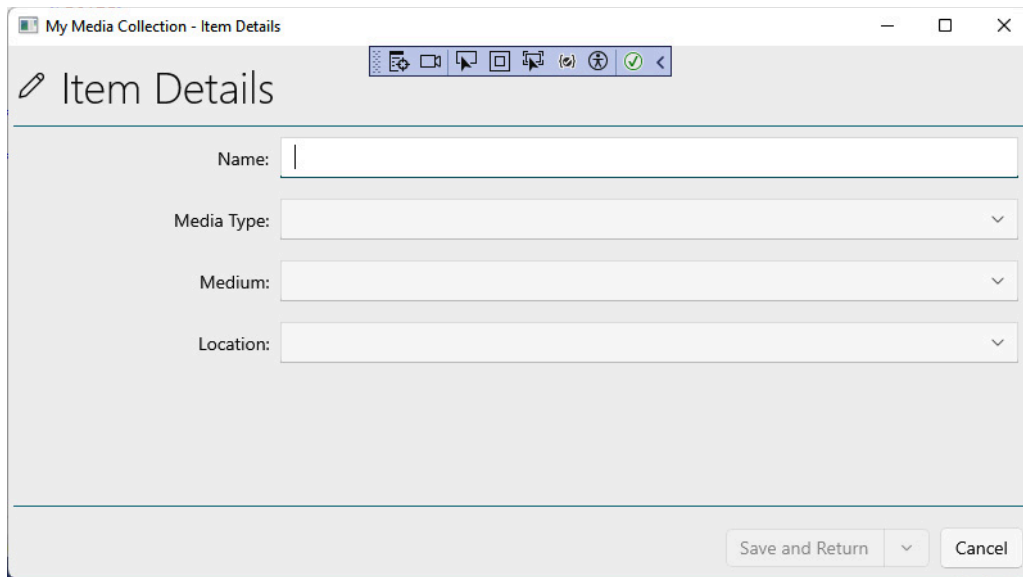


Figure 7.6 – The restyled Item Details page

This looks great. Now the styles of the two pages match, and the added border lines match the color of the highlighted active input field.

Let's now shift gears and review a tool that can help designers and developers when implementing Fluent Design.

Using the Fluent XAML Theme Editor

We've seen how easy it is to adopt the default color and theme resources from the user's Windows settings, but what if you or your company wants to create a custom theme for an application? Maybe that theme needs to be shared across a suite of applications. You could create a XAML file with **ResourceDictionary** in Visual Studio and code all the markup by hand for a new style. Visual Studio's **IntelliSense** will help in some regards. However, there is an easier way.

Microsoft has created an open source tool called the **Fluent XAML Theme Editor**. It is available on the Microsoft Store at <https://apps.microsoft.com/store/detail/9N2XD3Q8X57C>, and the source is available on GitHub at <https://github.com/Microsoft/fluent-xaml-theme-editor>. This application provides an easy-to-use visual designer to create a **ResourceDictionary** XAML file that you can drop into your projects.

NOTE

The Fluent XAML Theme Editor was created to adjust the styles for UWP controls, but these same styles will also work with WinUI 3 controls.

To install the application, open the **Microsoft Store** app, search for **fluent xaml** in the search field, and you will find **Fluent XAML Theme Editor** in the search results. Click it in the search results to view the product page:

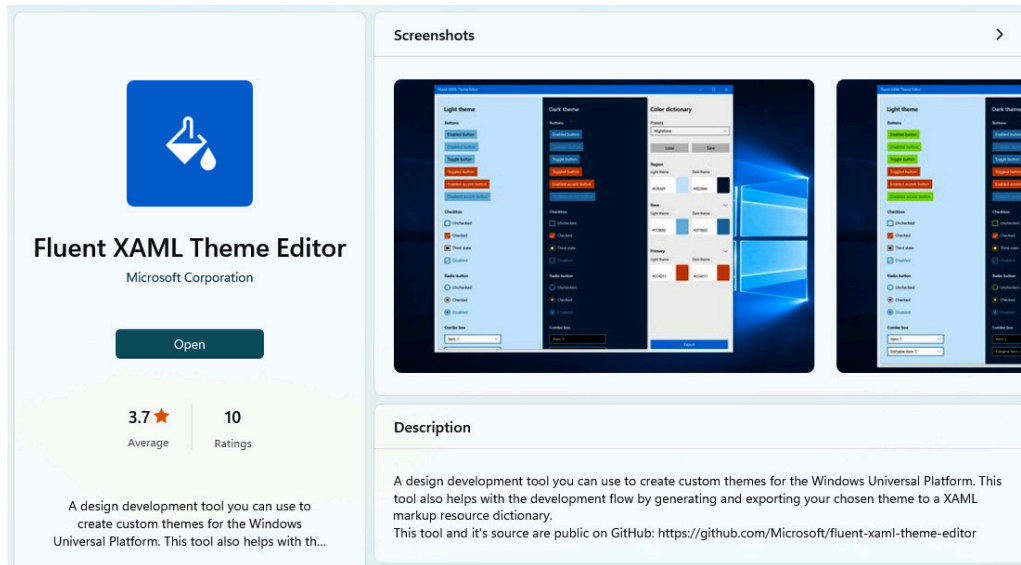


Figure 7.7 – The Fluent XAML Theme Editor page in the Microsoft Store

If you have already installed the app, there will be a **Launch** button. If it is not installed, you can click the **Install** button. When it completes its installation, you will find the app in your **Start** menu.

When you first launch the app, it will launch with the default styles for a UWP app, displayed in both light and dark themes. On the right-hand panel, you will find controls for changing the colors and shapes of UI elements. **Typography** is listed as **Coming Soon**, but it's been promising this for many years:

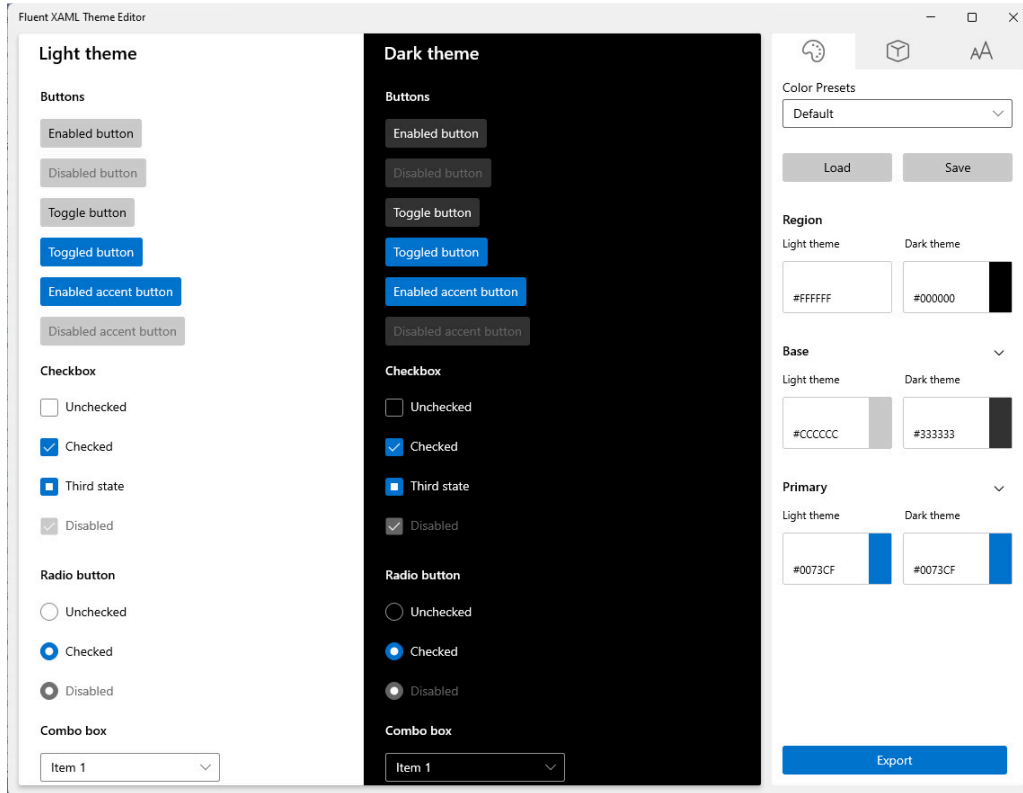


Figure 7.8 – The Fluent XAML Theme Editor for Windows

Colors

On the **Colors** tab, you can select one of the default profiles from the **Color Presets** dropdown. In addition to the default preset, there are **Lavender**, **Forest**, and **Nighttime** options. There are also options to load additional presets or to save your current color settings as a new preset. These color presets are saved in JSON format.

NOTE

Any colors you specify here will override the Windows system accent color that would otherwise be picked up by WinUI applications by default. Unless your application has a good reason to follow another theme, it's best to let WinUI use the user's chosen accent colors. Designing a custom theme should be undertaken by an experienced design team.

Clicking on any of the colors in the current preset will launch a color picker window, where you can adjust the current color:

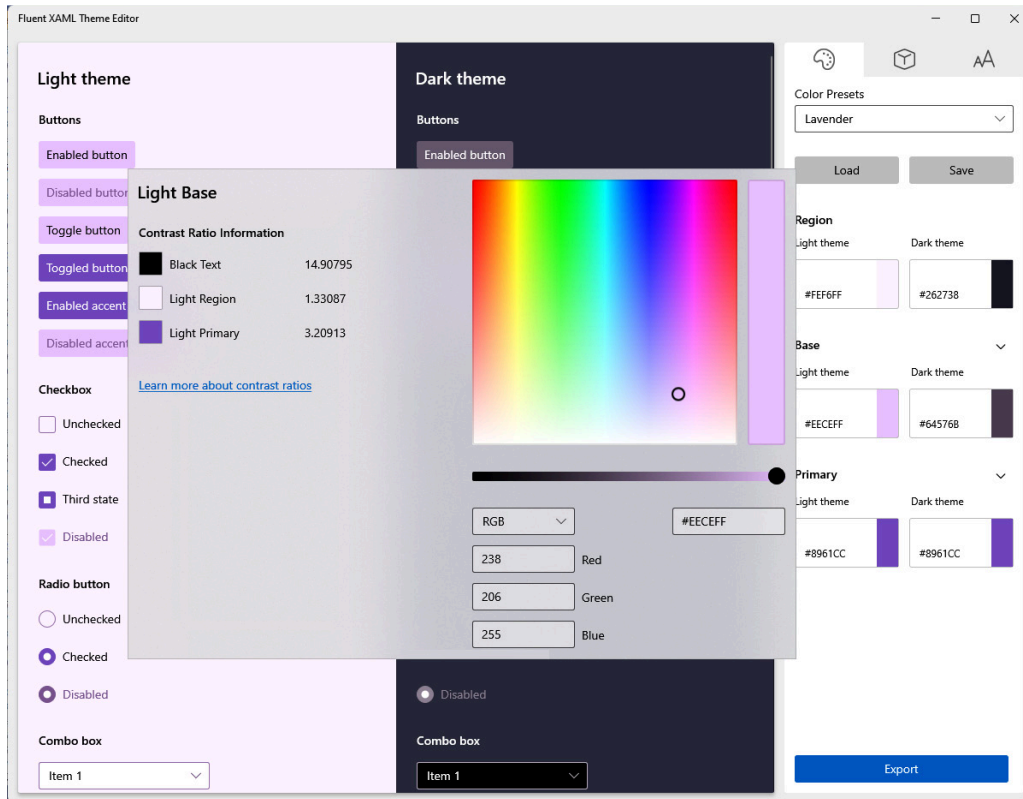


Figure 7.9 – Adjusting a preset color with a color picker

The **Region**, **Base**, and **Primary** colors can each be adjusted independently to change their light and dark theme appearance.

Shapes

The **Shapes** panel provides controls to adjust the **Corner Radius** of the **Controls** and the **Overlay**. This is also where you can adjust the default **Border Thickness** for the theme.

Shape presets, like colors, can also be saved and loaded. The app comes with two presets: **Default** and **No Rounding, Thicker Borders**. The difference is subtle but noticeable:

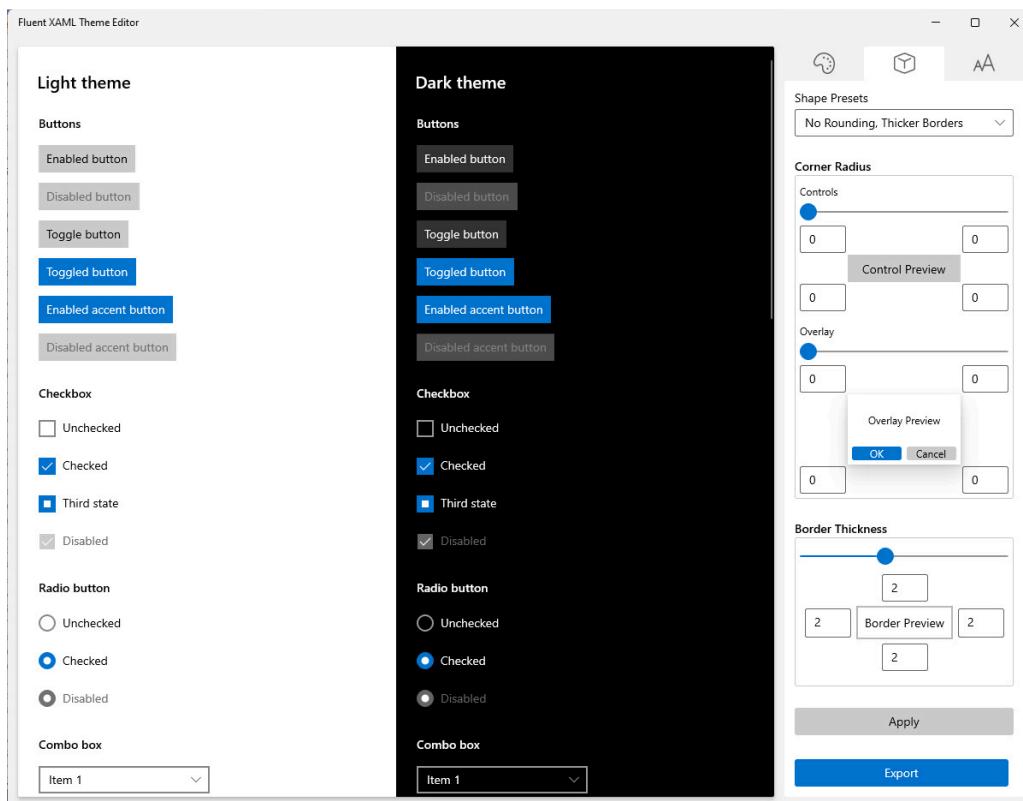


Figure 7.10 – Shapes with No Rounding and Thicker Borders applied

When you are done adjusting the color and shape settings, use the **Export** button to open a new window containing a **ResourceDictionary** with your theme data. You can copy the XAML and paste it into a **Resources** section in your project:

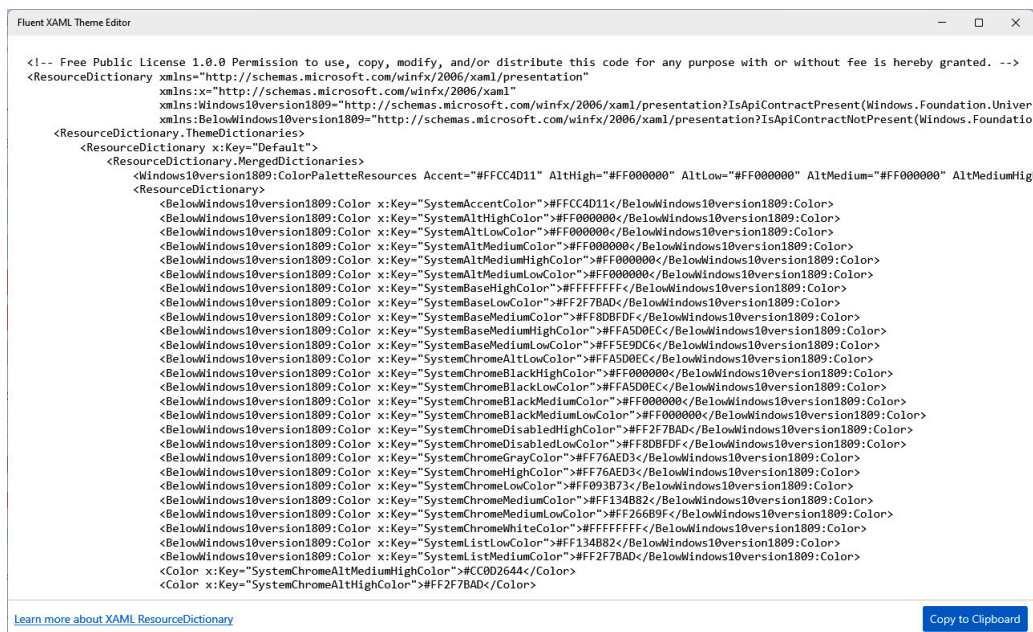


Figure 7.11 – Exporting a theme from the Fluent XAML Theme Editor

Next, let's explore the Acrylic material, which is part of the Fluent Design System that we touched on earlier in the chapter.

Acrylic material and the Fluent Design System

Acrylic is a WinUI brush that, when applied in your apps, provides a translucent texture. The texture works in both light and dark themes in Windows, and it is a great way to give users the feeling of depth when using your applications. The **AcrylicBrush** class is part of the **Microsoft.UI.Xaml.Media** namespace in the Windows App SDK, where **Brush**, **SolidColorBrush**, and **GradientBrush** are also found.

NOTE

The Acrylic material is also available to UWP applications that use WinUI 2.8.x.

If you want to explore the **AcrylicBrush** before deciding to use it in your own projects, you can try it in the WinUI 3 Gallery app:

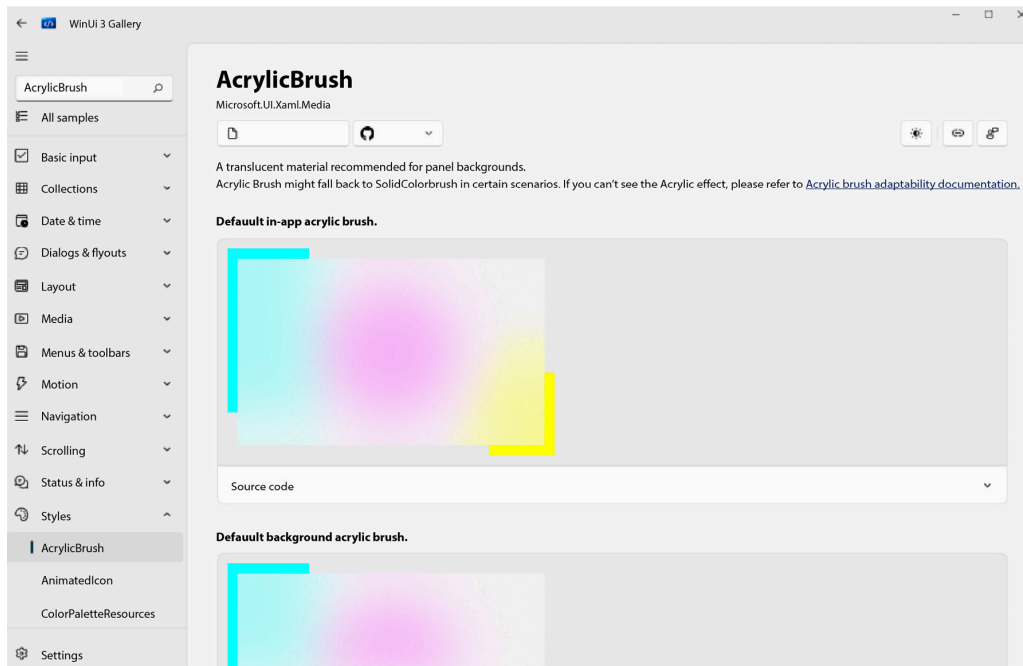


Figure 7.12 – Exploring AcrylicBrush in the WinUI 3 Gallery

On the gallery page, you can see the default styles of **AcrylicBrush** and how they'll appear in the Windows light and dark themes. Additionally, the gallery page provides controls to adjust the opacity and tint of the brush. You can also set a fallback solid color for the brush. The fallback color is used on systems without the resources to load an acrylic brush.

Depending on which brush is applied, the element will paint the acrylic brush based on background elements or in-app elements behind the current, overlapping element. The WinUI 3 Gallery has examples of how to apply either type of brush. This is an example of the in-app brush:

```
<Rectangle Fill="{ThemeResource AcrylicInAppFillColorDefaultBrush}"/>
```

This example shows how to apply a background acrylic brush:

```
<Rectangle Fill="{ThemeResource AcrylicBackgroundFillColorDefaultBrush}"/>
```

When incorporated into your WinUI application, an acrylic brush gives users the feeling of texture and depth, which are two of the fundamental principles of Fluent Design. Another material you can choose to incorporate is Mica.

Use Mica in WinUI applications

Mica is another material that's available to WinUI applications. You can think of Mica as an Acrylic background brush without the transparency. It creates the brush style based on the current desktop background color in Windows. There are two variants of the Mica material: **Mica** and **Mica Alt**. Mica Alt creates the same kind of opaque background as Mica, but it has stronger tinting.

NOTE

Mica is only available in Windows 11 and later. If your app uses Mica and is installed on Windows 10, it will not apply the material.

If you use the WinUI 3 Gallery and open the System Backdrops (Mica/Acrylic) page, you can launch a WinUI window and cycle through the Mica, Mica Alt, and Acrylic materials applied to the window background:

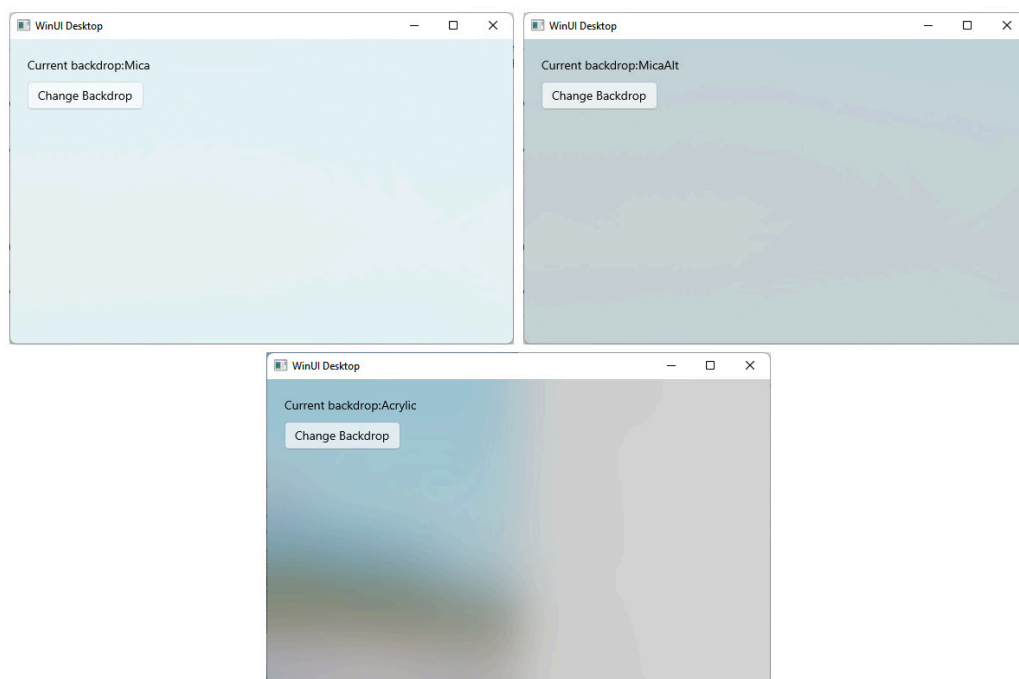


Figure 7.13 – Viewing the same window with Mica, Mica Alt, and Acrylic

The code to set the system backdrop for a window is more involved than just applying a brush to the **Fill** property of a **Rectangle** or another UI element. Let's try configuring Mica Alt in the My Media Collection app.

Incorporate Mica into My Media Collection

Bringing Mica into your own apps is easy with Windows App SDK 1.3 or later. Let's use the Mica Alt material in My Media Collection. It only takes a few steps:

1. First, check that your project is using Windows App SDK 1.3 or later. If it's not, you can use the **NuGet Package Manager** to upgrade it to the latest available version.
2. Next, open **MainPage.xaml** and **ItemDetailsPage.xaml** and remove the **Background** property for each **Page** element. If the **Page** elements (or any of their children) aren't transparent, you won't see the Mica backdrop.

3. Finally, open **MainWindow.xaml.cs** and add the following code to the constructor immediately after the call to **InitializeComponent**:

```
SystemBackdrop = new MicaBackdrop
{
    Kind = MicaKind.BaseAlt
};
```

That's it. Now run the app and see how the background looks with Mica. The color in the app will vary depending on your current Windows background:

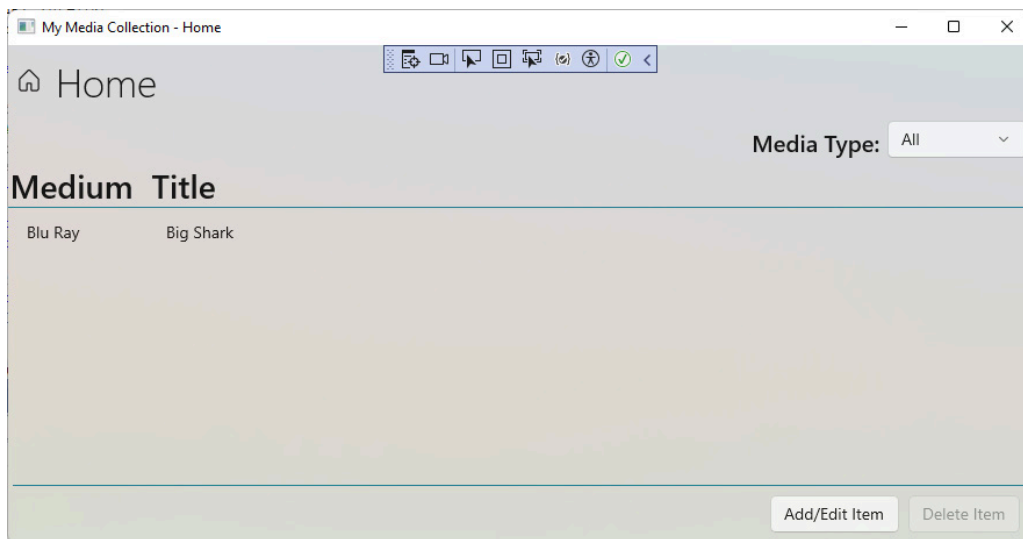


Figure 7.14 – Styling My Media Collection with Mica

Notice that the title bar does not pick up the Mica material, just as it ignores theme changes. If you would like to do this on your own, you can follow the Microsoft Learn instructions for extending your window contents into the title bar area:

<https://learn.microsoft.com/windows/apps/develop/title-bar?tabs=wasdk#full-customization>. For this approach, you will need to create your own title bar control with any buttons and icons and update the title bar text there during page navigation.

Now, we will take a quick look at some additional Fluent Design tools geared toward designers.

Design resources and toolkits for Fluent Design

While a deep dive into user interface design is beyond the scope of this book, we will briefly review some of the Fluent Design toolkits available for popular design tools. Design resources and examples for several of these tools can be downloaded from Microsoft Learn at

<https://learn.microsoft.com/windows/apps/design/downloads/>:

- **Figma:** This is a design and prototyping tool with free and paid options, depending on the team and project size. You can find out more about Figma on its website: <https://www.figma.com/>.
- **Sketch:** This is another popular tool for designing and prototyping applications individually or with teams. There is no free plan, but Sketch does have a free trial period. Sketch is available at <https://www.sketch.com/>.
- **Adobe XD:** XD is Adobe's design/prototype tool. Like Figma, Adobe XD has free and paid options for designing apps with their tool. Check out XD at <https://helpx.adobe.com/support/xd.html>.
- **Adobe Illustrator:** This is a powerful vector design tool from Adobe. There is a free trial available. Download and get started with Adobe Illustrator at <https://www.adobe.com/products/illustrator.html>.
- **Inkscape** (<https://inkscape.org/>) is a free vector image editor that can also work with **Adobe Illustrator (AI)** files.
- **Adobe Photoshop:** This is probably one of the best-known raster image editors. Adobe also has a free trial for Photoshop at <https://www.adobe.com/products/photoshop.html>.

The Fluent Design toolkit for Photoshop includes several PSD files. It is also possible to work with PSD files in free image editors such as **GIMP** (<https://www.gimp.org/>) or **Paint.NET** (<https://www.getpaint.net/>). Paint.NET requires an open source plugin, available at <https://www.psdplugin.com/>.

Summary

We have learned a lot about Fluent Design, design resources, and the tools available to WinUI developers in this chapter. You will be able to use these tools and techniques in your WinUI application design or recommend them to designers at your company. We also updated the My Media Collection app to be more compliant with Fluent Design recommendations and learned how to incorporate the Acrylic and Mica materials.

In the next chapter, we will examine how to add notifications to WinUI with the Windows App SDK notifications system.

Questions

1. Which platforms have Fluent Design implementations?
2. What is a control pattern?
3. Which font does Microsoft recommend using for Fluent Design?
4. Which aspect of style is specific to devices with large screens?
5. What are the names of the two spacing densities available in Fluent Design?
6. Which attribute can be set in **Application.xaml** to override a user's light/dark theme selection?
7. Which design tools have Fluent Design toolkits available?